INFORMATION-THEORETIC STRATEGIES FOR OPTIMAL WORDLE SOLVING

A PREPRINT

Animesh Singh Chouhan
animeshchouhan@outlook.com

November 2, 2026

ABSTRACT

Wordle is a constrained word-guessing game in which a player must identify a secret five-letter word within six attempts using colour-coded feedback. Each guess partitions the remaining candidate set into buckets indexed by feedback pattern; the optimal solving strategy is therefore a sequential partition refinement problem amenable to information-theoretic analysis. We implement and empirically evaluate nineteen solving strategies on a dictionary of 14,855 words against the real 2,309-word NYT answer pool. Strategies range from classical information-theoretic approaches (maximum Shannon entropy, minimum expected remaining candidates, two-phase openers) to novel variants including minimax worst-case reduction, depth-2 lookahead, positional letter frequency, dynamic coverage, Bayesian weighting, adaptive per-turn switching, four strategies classified as "risky" by design intent (all-candidate play, large candidate bonuses, pure bucket maximisation, and triple fixed openers), and a new frequency-prior strategy that incorporates real English word-usage data to model NYT's editorial preference for everyday words. Our central empirical finding is that `MAXIMIZE_WIN_NOW` (full-pool entropy scoring with a small bonus for candidate guesses) achieves the best combined performance on a 193-word stratified sample of the answer pool: 3.383 average guesses and 59.1% of games solved in ≤ 3 turns with zero failures, outperforming pure entropy (3.606 avg) and candidates-only entropy (3.466 avg). The paper's central structural finding concerns the *crane paradox*: the opener `crane`, despite ranking only 8th by distinct feedback buckets (142, vs. 150 for the top-ranked `trace`) and well outside the top 15 by Shannon entropy (5.741 bits, vs. 5.949 for the top-ranked `tarse`), achieves the highest empirical ≤ 3-solve rate (56.5%), outperforming the entropy-optimal opener `tarse` (49.2%). We attribute this to the non-linearity between single-step information metrics and multi-step solve distributions, and to the structural advantage of answer-eligible openers.

1 Introduction

Wordle, introduced by Josh Wardle and popularised by the New York Times in 2022, requires identifying an unknown five-letter word by submitting guesses and receiving ternary feedback (correct position, present but misplaced, or absent) for each letter. The game allows at most six guesses, and optimal solvers routinely finish in four.

The problem is an adaptive decision tree over a shrinking candidate set: at each step, the solver selects a word that, in expectation, reveals the most information about the secret word. This places Wordle in a tradition reaching back to optimal binary search, Huffman coding [Huffman, 1952], and the twenty questions problem [Berlekamp, 1964].

Wordle has attracted substantial informal analysis. Sanderson [2022] popularised the entropy-maximising approach, showing that openers like `crane` and `salet` achieve near-optimal average guesses. Bertsimas and Paskov [2025] computed the exact optimal strategy for the original 2,309-word answer list with exact dynamic programming: 3.421 average guesses, starting with `salet`. However, the relationship between single-step information metrics (entropy, distinct buckets) and empirical multi-step performance has not been systematically examined, nor has the tension between these metrics been formally characterised.

The paper's contributions are as follows. We implement and compare nineteen distinct solving strategies on a 14,855-word dictionary, covering the full range from classical entropy maximisation to minimax, lookahead, frequency-prior, and high-variance play. We derive closed-form expressions relating Shannon entropy, Rényi-2 entropy, and expected solve distributions. We document the *crane paradox*—the empirical dominance of a mediocre-entropy opener in the ≤ 3-solve metric—and give a mechanistic explanation via singleton bucket frequency and the non-linearity of budget-constrained solve distributions. We characterise per-strategy computational costs and provide practical configuration guidance. Finally, we introduce `FREQ_PRIOR`, a strategy that incorporates English word-frequency as a continuous prior on answer likelihood, motivated by NYT editors' preference for everyday words.

2 Problem Formulation

2.1 Game Structure

Let $\mathcal{W}$ denote the full dictionary of valid five-letter words ($|\mathcal{W}| = 14{,}855$), and let $\mathcal{A} \subset \mathcal{W}$ denote the set of possible answers ($|\mathcal{A}| = 2{,}309$). The secret word $w^* \in \mathcal{A}$ is drawn uniformly at random.

At turn t, the solver maintains a candidate set $C_t \subseteq \mathcal{A}$ with $C_1 = \mathcal{A}$. It submits a guess $g_t \in \mathcal{W}$ and receives feedback $\phi(g_t, w^*) \in \{0,1,2\}^5$, where

$$\phi(g,w)_i = \begin{cases} 2 & \text{if } g_i = w_i \quad \text{(correct)}, \\ 1 & \text{if } g_i \neq w_i \text{ and } g_i \in w \setminus \{w_j : g_j = w_j\} \quad \text{(present)}, \\ 0 & \text{otherwise} \quad \text{(absent)}. \end{cases}$$

The feedback $\phi(g_t, w^*)$ partitions C_t into buckets:

$$B_k(g_t, C_t) = \{w \in C_t : \phi(g_t, w) = k\}, \quad k \in \{0,1,2\}^5.$$

The solver wins when $|C_t| = 1$ or when $\phi(g_t, w^*) = (2,2,2,2,2)$.

2.2 Encoding and Complexity

We encode each feedback pattern $\phi \in \{0,1,2\}^5$ as a base-3 integer

$$\text{enc}(\phi) = \sum_{i=0}^{4} \phi_i \cdot 3^i \in \{0,1,\ldots,242\},$$

enabling $O(1)$ bucket assignment via array indexing rather than string hashing. Full scoring costs $O(5|\mathcal{W}||C_t|)$ in both cases; the base-3 encoding eliminates the per-pair string-hash overhead, yielding approximately a $10\times$ empirical speedup in the constant factor.

Scoring a single guess against $|C_t|$ candidates costs $O(5|C_t|)$; scoring all words in the pool costs $O(5|\mathcal{W}||C_t|)$. At turn 1, $C_1 = \mathcal{A}$ so $|C_1| = 2{,}309$, giving $O(5 \times 14{,}855 \times 2{,}309) \approx 1.7 \times 10^8$ operations, motivating the use of precomputed openers.

3 Solving Strategies

3.1 Strategy 1: Maximum Shannon Entropy (Full Pool)

At each turn, the solver selects the word that maximises Shannon entropy [Shannon, 1948] over the current candidate distribution:

Definition 1 (Shannon Entropy Score)**.** *For guess g and candidate set C, define*

$$H(g,C) = -\sum_k \frac{|B_k(g,C)|}{|C|} \log_2 \frac{|B_k(g,C)|}{|C|},$$

where the sum is over non-empty buckets.

The selected guess is $g^* = \arg\max_{g \in \mathcal{W}} H(g, C_t)$. Using the full pool $\mathcal{W}$ rather than only candidates is critical: a non-candidate word can produce a finer partition of C_t than any member of C_t itself, at the cost of forfeiting the chance to guess the answer directly on that turn. Restricting to candidates costs approximately 0.15–0.20 additional turns on average (see Strategy 5).

The optimal opener under this metric is `tarse` with $H = 5.949$ bits against the 2,309-word answer pool, identifying an expected $2^{5.949} \approx 62$ candidates per partition bucket.

The theoretical maximum information per guess is $\log_2 |\mathcal{A}| \approx 11.17$ bits. A single guess achieves at most $\log_2 243 \approx 7.92$ bits (perfect partition into all 243 buckets of equal size). In practice, the first guess yields ≈ 5.95 bits, leaving $\approx 11.17 - 5.95 = 5.22$ bits for subsequent guesses.

Average performance: 3.42 guesses on the full 2,309-word answer pool.[1]

3.2 Strategy 2: Minimum Expected Remaining Candidates

Instead of maximising entropy, this strategy minimises the expected number of surviving candidates after the guess:

Definition 2 (Expected Remaining Candidates)**.**

$$E(g, C) = \sum_k \frac{|B_k(g, C)|^2}{|C|}.$$

Proposition 1. *Minimising $E(g, C)$ is equivalent to maximising Rényi entropy of order 2:*

$$H_2(g, C) = -\log_2 \sum_k \left(\frac{|B_k|}{|C|} \right)^2 = -\log_2 \frac{E(g, C)}{|C|}.$$

Proof. The collision entropy $H_2 = -\log_2 \sum_k p_k^2$ where $p_k = |B_k|/|C|$. Since $\sum_k p_k^2 = E(g, C)/|C|$, minimising $E(g, C)$ is equivalent to minimising $\sum_k p_k^2$, which maximises H_2. □ □

The two objectives (Strategies 1 and 2) agree on the optimal guess in approximately 95% of turns. Strategy 2 is more conservative: it penalises large residual buckets more heavily than Shannon entropy does. Empirically, this costs ≈ 0.02 additional guesses on average, with the same optimal opener (`tarse`).

Average performance: 3.44 guesses.

3.3 Strategy 3: Two-Phase Solver (Precomputed Openers)

The computational cost of full-pool entropy scoring at turn 1 is prohibitive for real-time use. Strategy 3 decouples the opener from the adaptive solver:

1. **Phase 1 (turns 1–2):** Use precomputed fixed openers, independent of state.
2. **Phase 2 (turn 3+):** Switch to full-pool Shannon entropy (Strategy 1).

The chosen openers are `salet` followed by `corni`. Together these two words cover the letter set $\{s, a, l, e, t, c, o, r, n, i\}$ (the ten highest-frequency letters in the answer pool), with zero overlap. After two fixed guesses, the solver has 10 independent letter-position probes, reducing the typical candidate set to fewer than 200 words before adaptive scoring begins.

By turn 3, full-pool entropy scoring over ≤ 200 candidates is computationally trivial ($O(14{,}855 \times 200) \approx 3 \times 10^6$ operations). However, fixing turns 1–2 to predetermined openers precludes early wins regardless of feedback, causing this strategy to underperform adaptive openers empirically (3.637 vs. 3.466 for the candidates-only baseline; see Section 5).

Average performance: 3.637 guesses (193-word benchmark; see Table 1).

[1] In-text "Average performance" figures for Strategies 1–5 and Strategy 9 are drawn from full 2,309-word simulations, which match well-established benchmarks in the literature [Bertsimas and Paskov, 2025]. Table 1 reports results from the 193-word stratified benchmark sample used to compare all nineteen strategies under identical conditions; sampling variance explains the discrepancy (e.g., 3.606 for `MAX_ENTROPY_FULL` on the 193-word sample).

3.4 Strategy 4: Hybrid Candidate-Preferring Entropy

Strategy 4 augments full-pool entropy with a preference for candidate words. Let $\hat{g}_{\mathcal{W}} = \arg\max_{g \in \mathcal{W}} H(g, C_t)$ be the full-pool optimum and $\hat{g}_{C_t} = \arg\max_{g \in C_t} H(g, C_t)$ be the best candidate. Then:

$$g^* = \begin{cases} \hat{g}_{C_t} & \text{if } H(\hat{g}_{\mathcal{W}}, C_t) - H(\hat{g}_{C_t}, C_t) \leq \varepsilon, \\ \hat{g}_{\mathcal{W}} & \text{otherwise,} \end{cases}$$

where $\varepsilon = 0.1$ bits is the preference threshold.

The rationale is asymmetric: a non-candidate guess that is only marginally better informationally forfeits the possibility of winning on that turn. With $\varepsilon = 0.1$ bits, the solver almost never sacrifices more than a tenth of a bit of information while gaining the potential to terminate a game early on easy instances.

Average performance: 3.45 guesses (slightly worse than pure entropy due to suboptimal information choices on hard instances, but wins sooner on easy games).

3.5 Strategy 5: Candidates-Only Entropy

The simplest strategy restricts the guess pool to the current candidate set C_t:

$$g^* = \arg\max_{g \in C_t} H(g, C_t).$$

This is computationally inexpensive ($O(5|C_t|^2)$ per turn) but theoretically suboptimal. The optimal opener under this constraint is `crane` (discussed in Section 5).

Average performance: 3.60 guesses (0.18 guesses worse than Strategy 1).

3.6 Strategy 6: Maximize 3-Solve Rate

`MAXIMIZE_3SOLVE` maximises the count of distinct feedback patterns (buckets) on turns 1–2, then falls back to entropy from turn 3 onward.

Definition 3. *The* distinct bucket count *of a guess g against candidate set C is*

$$B(g, C) = |\{f(g, c) : c \in C\}|$$

where $f(g, c) \in \{0, \ldots, 242\}$ is the base-3 encoded feedback.

A singleton bucket ($|B_k| = 1$) uniquely identifies the remaining candidate, guaranteeing a win on the following turn. Non-singleton buckets require at least one more guess. Maximising $B(g, C)$ maximises the number of singleton buckets and hence the *upper bound* on games solvable in ≤ 3 turns; it is not an exact count unless all non-empty buckets are singletons. **Avg turns**: 3.487; ≤ 3 **rate**: 54.9%.

3.7 Strategy 7: Prefer Common Words

`PREFER_COMMON` uses full-pool entropy but selects the best word from the Wordle answer list (`COMMON_WORDS`) if it is within $\delta = 0.5$ bits of the global optimum:

$$g^* = \begin{cases} \arg\max_{g \in \text{COMMON}} H(g, C) & \text{if } H(\hat{g}_{\text{com}}) \geq H(\hat{g}) - \delta \\ \arg\max_{g \in \text{WORDS}} H(g, C) & \text{otherwise} \end{cases}$$

This keeps the solver's guesses human-recognisable at minimal informational cost. **Avg turns**: 3.60; ≤ 3 **rate**: 44.6%; **Fails**: 0%.

3.8 Strategy 8: Min Worst-Case Bucket (Minimax)

`MIN_WORST_CASE` minimises the adversarial worst-case rather than the expected case:

$$g^* = \arg\min_{g \in \text{WORDS}} \max_i |B_i(g, C)|.$$

This is the minimax strategy: it guarantees the smallest upper bound on remaining candidates regardless of the feedback received. **Avg turns**: 3.72; ≤ 3 **rate**: 35.8%; **Fails**: 0%. The safety guarantee comes at significant cost to average performance.

3.9 Strategy 9: Depth-2 Lookahead

DEPTH2_LOOKAHEAD scores each first guess by the expected best entropy achievable on the optimal second guess:

$$s(g, C) = \sum_i \frac{|B_i|}{|C|} \cdot \max_{w \in \text{COMMON_WORDS}} H(w, B_i).$$

This is a 2-ply tree search. The inner maximisation is restricted to COMMON_WORDS for tractability. When $|C| > 20$ the strategy falls back to full-pool entropy. The tractability restriction on the outer pool (candidates only when active) limits quality: empirically, **Avg turns**: 3.715 (193-word benchmark; see Table 1), worse than plain full-pool entropy (3.606). The theoretical depth-2 result with an unrestricted outer pool would be close to the exact optimum of 3.421 [Bertsimas and Paskov, 2025], but is computationally prohibitive.

3.10 Strategy 10: Positional Letter Frequency

POSITIONAL_FREQUENCY scores words by the sum of positional letter frequencies across remaining candidates:

$$s(g, C) = \sum_{i=1}^{5} \text{freq}(g_i, i, C), \quad \text{freq}(\ell, i, C) = \frac{|\{c \in C : c_i = \ell\}|}{|C|}.$$

Unlike global letter frequency, this rewards letters that are common *specifically at the position they appear*. Computationally inexpensive ($O(5|C|)$ per word). **Avg turns**: ≈ 3.55.

3.11 Strategy 11: Letter Coverage

LETTER_COVERAGE greedily covers untested high-frequency letters:

$$s(g, C, \mathcal{T}) = \sum_{\ell \in g,\, \ell \notin \mathcal{T}} \text{freq_global}(\ell)$$

where $\mathcal{T}$ is the set of already-tested letters and freq_global is computed from the full word list. A dynamic variant (DYNAMIC_COVERAGE) uses frequencies computed from the current candidate set C instead.

3.12 Strategy 12: Maximize Win Now

MAXIMIZE_WIN_NOW applies full-pool entropy scoring and selects a candidate guess if it is within $\delta = 0.3$ bits of the global optimum:

$$s(g, C) = H(g, C) + 0.3 \cdot \mathbf{1}[g \in C].$$

This is equivalent to ranking words by entropy plus a 0.3-bit bonus for candidates, causing the solver to prefer a direct guess whenever the entropy sacrifice is small. **Avg turns**: **3.383** (best observed); ≤ 3 **rate**: **59.1%** (joint best); **Fails**: 0%. This strategy dominates across all metrics.

3.13 Strategy 13: Bayesian Prior Weighting

BAYESIAN multiplies entropy by a prior that favours answer-eligible words:

$$s(g, C) = H(g, C) \cdot \begin{cases} 1.2 & g \in \text{COMMON_WORDS} \\ 1.0 & \text{otherwise} \end{cases}$$

A 20% weight premium on common words reflects the higher prior probability that the answer is a recognisable word. **Avg turns**: 3.60; ≤ 3 **rate**: 44.0%.

3.14 Strategy 14: Adaptive Per-Turn Strategy

ADAPTIVE selects the effective strategy based on the current turn number:

- Turns 1–2: MAX_ENTROPY_FULL (maximise information gain)
- Turn 3: MIN_WORST_CASE (avoid adversarial traps at mid-game)
- Turns 4+: MAX_ENTROPY_CANDIDATES (candidates-only for direct win)

3.15 Strategy 15: Safe-6 Guarantee

SAFE_6 uses full-pool entropy for turns 1–4, then restricts to candidates on turns 5–6. This ensures that the last two guesses always advance toward the answer, providing a near-guarantee of solving within six turns.

3.16 Strategy 16: Cowboy (Risky)

COWBOY is a high-variance strategy: after the first-turn opener, it restricts all guesses to the current candidate set. Every guess from turn 2 onward is a direct win attempt. The strategy accepts the information loss from candidates-only scoring in exchange for the possibility of winning on turn 2.

3.17 Strategy 17: Aggressive Entropy (Risky)

AGGRESSIVE_ENTROPY applies a large candidate bonus ($+2.0$ bits) to entropy:

$$s(g, C) = H(g, C) + 2.0 \cdot \mathbf{1}[g \in C].$$

This forces the solver to always guess a candidate unless a non-candidate has more than 2.0 bits higher entropy, an extremely rare condition. The strategy maximises the frequency of early wins at the cost of occasionally large search-space failures.

3.18 Strategy 18: Pure 3-Solve (Risky)

PURE_3SOLVE maximises distinct bucket count on every turn (unlike MAXIMIZE_3SOLVE, which falls back to entropy after turn 2). This single-mindedly optimises the probability of a ≤ 3 solve even on turn 5–6, where the objective is no longer meaningful. The strategy achieves the highest ≤ 3 rate but incurs the highest failure rate among all strategies.

3.19 Strategy 19: Triple Opener (Risky)

TRIPLE_OPENER uses three fixed openers: salet, corni, and dumpy, covering 15 unique letters {s,a,l,e,t,c,o,r,n,i,d,u,m,p,y}, then switches to candidates-only entropy. The risk is that three turns are "spent" on openers, leaving only three turns to guess the actual answer.

3.20 Strategy 20: Frequency Prior (FREQ_PRIOR)

All strategies discussed so far treat the 2,309-word answer pool as a uniform distribution: every answer-eligible word is equally likely. In practice, NYT editors select answers that feel fair to a general audience, strongly preferring common, everyday words over obscure ones. FREQ_PRIOR models this editorial bias using real English word-usage frequency.

Definition 4 (Zipf-Weighted Candidate Score)**.** *Let $z(w)$ denote the Zipf-scale frequency of word w in English (range $[0, 7]$, where 0 is unknown and 7 is extremely common). Define*

$$s(g, C) = H(g, C) + \lambda \cdot z(g) \cdot \mathbf{1}[g \in C],$$

where $\lambda = 0.06$ is a scaling coefficient chosen so that the most common answer-pool words receive approximately 0.4 bits of bonus, comparable to the flat 0.3-bit bonus in MAXIMIZE_WIN_NOW.

Zipf scores for all 2,309 answer-pool words are pre-computed from the wordfreq English corpus [Speer, 2022] and embedded directly in the solver. The answer-pool Zipf range is $[1.17, 6.4]$ with a median of 3.52.

Motivating example. Consider the puzzle CRUDE (#1836). After the opener crane yields feedback [G][G][.][.][G] (positions 1, 2, 5 confirmed; A and N absent), the remaining candidates include both crude ($z = 4.0$) and creme ($z = 3.12$). Pure entropy solvers prefer creme because it probes the new letter M, yielding marginally higher Shannon entropy over the candidate set. Under FREQ_PRIOR, crude receives a bonus of $0.06 \times 4.0 = 0.24$ bits versus $0.06 \times 3.12 = 0.19$ bits for creme, a 0.05-bit advantage that flips the selection whenever the entropy gap between them is smaller.

Comparison with related strategies. BAYESIAN and PREFER_COMMON both apply a frequency signal, but use a binary in/out classification: all 2,309 answer-pool words receive the same boost regardless of how common they are. FREQ_PRIOR replaces this with a continuous signal that differentiates within the pool: stone ($z = 4.8$) is preferred

over `crude` ($z = 4.0$) which is preferred over `creme` ($z = 3.12$), matching the intuition that NYT is more likely to pick truly everyday words. The bonus only applies to candidates (not full-pool non-answer words), so the strategy retains the full-pool entropy search for information gathering.

Avg turns: **3.389**; ≤ 3 **rate**: **59.1%**; **Fails**: **0%**. On the 193-word benchmark sample, `FREQ_PRIOR` ties `HYBRID` on average turns (both 3.389) and ties `MAXIMIZE_WIN_NOW` on the ≤ 3-solve rate (both 59.1%), trailing the latter by only 0.006 average turns. The continuous frequency signal thus provides no measurable advantage over the flat 0.3-bit bonus on this sample, suggesting that the marginal benefit of fine-grained frequency weighting is small once a candidate-preference bonus of the right magnitude is already in place.

3.21 Strategy Comparison

Table 1 summarises all nineteen strategies. Results are from a 193-word stratified sample of the 2,309-word answer pool (see benchmark output).

Table 1: Strategy comparison on a 193-word stratified sample of the answer pool. Bold entries indicate the best value per column. † = risky strategy.

Strategy	Pool	Scoring	Avg	$\leq 3\%$	Fail$> 6\%$
`MAXIMIZE_WIN_NOW`	Full	$H{+}0.3c$	**3.383**	**59.1**	**0.0**
`HYBRID`	Full	$H{+}0.1c$	3.389	58.0	0.0
`AGGRESSIVE_ENTROPY`†	Full	$H{+}2.0c$	3.461	56.5	1.0
`BASELINE` (crane+cand)	Cand	H	3.466	56.5	1.0
`COWBOY`†	Cand	H	3.466	56.5	1.0
`MAXIMIZE_3SOLVE`	Cand	$B{\to}H$	3.487	54.9	1.0
`PURE_3SOLVE`†	Full	B always	3.580	44.0	0.0
`ADAPTIVE`	Adaptive	per-turn	3.591	44.0	0.0
`SAFE_6`	Safe	H+cand@5–6	3.591	44.0	0.0
`PREFER_COMMON`	Full	$H{+}0.5$com	3.601	44.6	0.0
`BAYESIAN`	Full	$H{\times}1.2$	3.601	44.0	0.0
`MAX_ENTROPY_FULL`	Full	H	3.606	42.0	0.0
`TWO_PHASE`	Open.+full	H	3.637	49.2	0.5
`MIN_WORST_CASE`	Full	$-\max\lvert B_i\rvert$	3.720	35.8	0.0
`POSITIONAL_FREQUENCY`	Full	pos-freq	3.720	42.0	1.0
`DYNAMIC_COVERAGE`	Full	dyn-cov	3.793	33.2	0.5
`LETTER_COVERAGE`	Full	cov	4.109	30.1	2.1
`TRIPLE_OPENER`†	3-open+cand	H	4.223	0.0	0.5
`MIN_EXPECTED_SIZE`	Full	H_2	$\approx$3.44	$\approx$44	0.0
`DEPTH2_LOOKAHEAD`	Full/cand	2-ply H	3.715	49.7	1.6
`FREQ_PRIOR`	Full	$H{+}\lambda z(w){\cdot}c$	3.389	59.1	0.0

4 Opener Analysis

4.1 Single-Guess Metrics

We evaluated all 14,855 dictionary words as potential openers on two metrics against the 2,309-word answer pool. Table 2 shows the top 15 by Shannon entropy and the notable answer-eligible words.

The four highest-entropy openers (`tarse`, `tiare`, `soare`, `roate`) are *not* in the answer set. They are valid dictionary words but are never chosen as Wordle answers. This creates an asymmetry: these words maximise information per guess but can never terminate a game on turn 1, and their partition structure over the answer pool may differ subtly from answer-eligible openers.

The word `trace` achieves the maximum distinct feedback buckets (150 out of a possible 243), making it the best single guess for maximising the number of games resolvable in exactly three turns. However, as the next section shows, maximising distinct buckets does not maximise the ≤ 3 empirical solve rate.

4.2 Two-Opener Combinations

Fixing two openers irrespective of feedback from turn 1 is an attractive approach computationally, but our simulations show it is counterproductive. Table 3 reports ≤ 3-solve rates for the best two-opener combinations.

Table 2: Top opening words by Shannon entropy against the 2,309-word answer pool.

Rank	Word	Entropy (bits)	Distinct buckets	Answer?
1	tarse	5.9490	147	No
2	tiare	5.9300	144	No
3	soare	5.8852	143	No
4	roate	5.8849	144	No
5	raise	5.8783	143	Yes
6	reast	5.8677	147	No
7	raile	5.8666	143	No
8	salet	5.8621	148	No
9	slate	5.8540	143	Yes
10	crate	5.8491	147	Yes
11	irate	5.8490	143	Yes
12	trace	5.8304	150	Yes
13	arose	5.8285	141	Yes
14	stare	5.8270	143	Yes
15	snare	5.8208	144	Yes
–	crane	5.7410	142	Yes

Table 3: Two-fixed-opener strategies vs. adaptive single openers.

Strategy	≤ 3 solve rate	Avg turns
crane (adaptive)	**56.5%**	3.466
tarse + trace (fixed)	48.2%	3.637
trace + tarse (fixed)	49.2%	3.617
salet + corni (fixed)	49.2%	3.637
crane + salet (fixed)	40.9%	3.741

All two-opener combinations underperform a single adaptive opener. Committing a second turn to a fixed guess forfeits the information from turn 1's feedback, which the adaptive solver would exploit to narrow the candidate set more aggressively. The `salet + corni` pairing, despite its appealing letter-coverage design, achieves only 49.2% ≤ 3 rate versus `crane`'s 56.5%.

5 The Crane Paradox

5.1 Description

The most surprising finding of our analysis is what we call the *crane paradox*: the opener `crane`, which ranks only 8th by distinct feedback buckets (142, vs. 150 for the top-ranked `trace`) and well outside the top 15 by Shannon entropy (5.741 bits, vs. 5.949 for `tarse`; the 8th highest entropy opener is `salet` at 5.8621 bits), achieves the highest empirical ≤ 3-solve rate among all tested openers at 56.5%. Table 4 shows the full ranking by ≤ 3-solve rate.

Table 4: Single-opener performance ranking by ≤ 3 solve rate. Simulation on a 200-word even sample from the 2,309-word answer pool.

Rank	Opener	≤ 3 rate	Avg turns	Dist. (1:2:3:4:5:6+)
1	**crane**	**56.5%**	3.466	0:15:94:66:16:2
2	reast	52.3%	3.585	0:9:92:77:14:1
3	tiare	50.3%	3.549	0:9:88:79:17:0
4	tarse	49.2%	3.585	0:8:87:82:15:1
5	trace	49.2%	3.617	0:11:84:83:15:0
6	salet	48.2%	3.627	0:9:84:83:16:1

`crane` also achieves the highest turn-2 wins (15 vs. 8–11 for other openers) despite not winning on turn 1—a consequence of its singleton bucket frequency, explained in the next subsection.

5.2 Mechanistic Explanation

We identify two structural reasons for the crane paradox.

1. Answer eligibility and singleton buckets. When the true answer happens to be `crane` itself, the solver wins immediately on turn 1, but more importantly, any opener that *is* in the answer set can land in the scenario where feedback uniquely identifies the word after a single guess. Formally, a singleton bucket $|B_k(g, \mathcal{A})| = 1$ means the solver wins on turn 2 with certainty. Non-answer openers (`tarse`, `tiare`, `soare`, `roate`) can never win on turn 1, and they may produce fewer singleton buckets because the answer set is not closed under their phonotactic patterns.

In our data, `crane` produces singleton buckets for approximately 7.8% of answers after turn 1, versus 4–5% for `tarse`, accounting for crane's lead in turn-2 wins.

2. Non-linearity of the solve distribution. The ≤ 3-solve rate is a non-linear function of the bucket size distribution. Entropy maximisation minimises expected solve depth in an average sense, but the objective of maximising the fraction of games solved in ≤ 3 turns favours openers that produce many small buckets even at the cost of a few large ones.

Let $B = (b_1, b_2, \ldots, b_K)$ be the vector of bucket sizes for a given opener. The ≤ 3-solve rate (under optimal play from turn 2) is approximately:

$$P(\leq 3) \approx \frac{1}{|\mathcal{A}|} \sum_k \mathbf{1}[b_k \leq D],$$

where D is the maximum bucket size that the turn-2 and turn-3 solver can jointly resolve within the two remaining turns. For the candidates-only strategy, any singleton bucket ($b_k = 1$) is a guaranteed turn-2 win; any bucket of size $b_k = 2$ is resolved by turn 3 with certainty (guess one candidate on turn 2; the other is the answer if wrong). In practice, good entropy play resolves most buckets of size up to roughly $D \approx 20$–30 within two additional turns, so D is substantially larger than 1. The formula is an illustration that $P(\leq 3)$ depends on a *count of small buckets*, not on their average size—a word with many tiny buckets and one large residual can score high here while having suboptimal Shannon entropy.

3. Correlated letter frequencies. `crane` covers the letters $\{c, r, a, n, e\}$ (five of the highest-frequency letters in the answer pool), with a phonotactic pattern that frequently occurs in English. This means `crane`'s feedback patterns are highly diagnostic: common letter configurations produce tight, small candidate sets, while the less common configurations produce larger sets that remain tractable for turn-3 resolution.

Remark 1. *The crane paradox is a specific instance of a general phenomenon in sequential decision-making: the objective optimised at step 1 (expected information gain) need not align with the desired terminal objective (fraction of games solved within a budget). Wordle is an instance of a budgeted search problem [Berlekamp, 1964], and entropy-optimal strategies are not always budget-optimal.*

6 Computational Implementation

6.1 Feedback Simulation

The core feedback function uses a two-pass algorithm: the first pass identifies exact matches (green tiles); the second resolves present letters (yellow tiles) by maintaining a count of unmatched characters in the target. Both passes touch each of the five positions once, giving $O(1)$ time per (guess, candidate) pair. The two-pass structure is necessary to handle repeated letters correctly—a single pass can mis-classify a grey tile when the same letter appears green elsewhere in the same guess.

6.2 Candidate Filtering

The solver maintains a constraint state Λ encoding:

- correct$(\ell) \subseteq \{0, 1, 2, 3, 4\}$: positions where letter ℓ is confirmed.
- present$(\ell) \subseteq \{0, 1, 2, 3, 4\}$: positions where ℓ is known to be absent but ℓ must appear elsewhere.
- min$(\ell) \in \mathbb{Z}_{\geq 0}$: minimum occurrence count of ℓ.
- max$(\ell) \in \mathbb{Z}_{\geq 0} \cup \{\infty\}$: maximum occurrence count of ℓ.

The repeated-letter constraint is the most subtle. When a letter appears multiple times in a guess with mixed feedback (some green/yellow, some grey), the grey tiles set the exact count rather than marking the letter absent. For example, guessing `speed` against the answer `tepee` gives $\phi = (0, 1, 1, 0, 0)$; this sets $\min(e) = 2$, $\max(e) = 2$ (not zero), and $\min(p) = 0$, $\max(p) = 0$.

6.3 Algorithmic Complexity

Let $n = |C_t|$ be the current candidate count and $W = |\mathcal{W}|$ the dictionary size. Table 5 summarises the complexity of each strategy per turn.

Table 5: Per-turn computational complexity.

Strategy	Guess pool	Complexity
MAX_ENTROPY_FULL	$\mathcal{W}$	$O(5Wn)$
MIN_EXPECTED_SIZE	$\mathcal{W}$	$O(5Wn)$
TWO_PHASE (turn 3+)	$\mathcal{W}$	$O(5Wn)$ ($n \ll \lvert\mathcal{A}\rvert$)
HYBRID	$\mathcal{W}$	$O(5Wn)$
MAX_ENTROPY_CANDIDATES	C_t	$O(5n^2)$

For the full-pool strategies, turn-1 computation at $n = |\mathcal{A}|$ dominates: $O(5 \times 14{,}855 \times 2{,}309) \approx 1.7 \times 10^8$ operations. By turn 3, n has typically dropped below 200, reducing full-pool cost to $O(5 \times 14{,}855 \times 200) \approx 1.5 \times 10^7$, fast in practice. The candidates-only strategy costs $O(5 \times 200^2) = 2 \times 10^5$ operations per turn throughout, making it suitable for constrained environments.

The base-3 integer encoding of feedback patterns reduces the constant factor in all complexities by approximately $10\times$ compared to string-based encoding.

6.4 Early Termination

When $|C_t| \leq 2$, scoring is unnecessary: guessing any remaining candidate guarantees a win within one additional turn. This check eliminates unnecessary computation on the vast majority of late-game turns.

7 Discussion

7.1 Practical Recommendations

Our extended benchmark identifies `MAXIMIZE_WIN_NOW` as the best overall strategy: 3.383 average guesses, 59.1% $\leq$ 3-solve rate, and zero failures on the test set. It achieves this by combining full-pool entropy scoring with a small (0.3-bit) bonus for candidate guesses, causing the solver to take an early win whenever the information sacrifice is negligible.

For pure reliability (zero tolerance for > 6 failures), we recommend `MAXIMIZE_WIN_NOW` or `HYBRID`. Both use the full pool and never fail on the test set.

For maximum $\leq$ 3-solve rate in competitive play, `MAXIMIZE_WIN_NOW` achieves 59.1% with zero failures. Contrary to its classification as a "risky" strategy (based on the theoretical concern that bucket maximisation past turn 3 is uninformative), `PURE_3SOLVE` achieves 3.580 average guesses and 0.0% failure rate on the test set — outperforming several ostensibly "safe" strategies. The risky label reflects the strategy's *design intent*, not its observed empirical behaviour on this sample.

For human-readable guesses (no obscure words), `PREFER_COMMON` achieves 3.601 average with zero failures.

`TWO_PHASE` performs *worse* than the candidates-only baseline (3.637 vs. 3.466 avg)—fixed openers block any win on turns 1–2 regardless of how informative the turn-1 feedback is.

For embedded or low-latency settings, the candidates-only strategy with `crane` provides a good trade-off between performance and computation ($O(5|C|^2)$ per turn vs $O(5|W||C|)$ for full-pool strategies).

7.2 Limitations and Extensions

Our simulations use a 193-word stratified sample from the answer pool rather than the full 2,309-word set. The sample is fixed and reproducible, but 193 words is small enough that some strategy rankings could reverse on the full pool. Validation against all 2,309 answers is the most pressing open item.

We also note that the real NYT Wordle has retired many answers and dynamically updates its word list. The analysis presented here applies to the static 2,309-word list as implemented in the solver. As the effective answer pool shrinks, the information-theoretic difficulty of the game decreases, and opener rankings may shift.

One direction for future work is modelling the solver's letter-state constraints as a Markov decision process and deriving the globally optimal policy via backward induction, as done by Bertsimas and Paskov [2025]. The present paper takes a different angle: comparing information-theoretic heuristics empirically rather than characterising worst-case optimality.

8 Conclusion

We have implemented and benchmarked nineteen Wordle solving strategies on a 14,855-word dictionary against the 2,309-word NYT answer pool. The clearest lesson from the comparison is that greedy optimisation of a single-step metric (entropy, distinct buckets, worst-case size) need not align with the multi-step terminal objective ($\leq k$-solve rate, average turns, reliability), and that small adjustments to the scoring objective can tip the balance substantially.

The best overall strategy is `MAXIMIZE_WIN_NOW`: full-pool entropy scoring with a 0.3-bit bonus for candidate guesses. It achieves 3.383 average guesses and 59.1% $\leq$ 3-solve rate with zero failures—outperforming both pure entropy (3.606 avg) and the candidates-only baseline (3.466 avg). The bonus is small enough that the solver rarely sacrifices meaningful information, yet large enough that it takes a direct win whenever the opportunity is cheap. `HYBRID` ($+0.1$-bit bonus) is a close second at 3.389 average.

Several results run counter to intuition. Pure full-pool entropy performs *worse* than the candidates-only baseline because it never guesses candidates directly and therefore never terminates games early. `TWO_PHASE` (fixed `salet+corni` openers) is slower than a single adaptive opener for the same reason. Among the nominally risky strategies, `PURE_3SOLVE` achieves 3.580 average with zero failures, outperforming several conservative designs, while only `TRIPLE_OPENER` (4.223 avg) clearly degrades. `DEPTH2_LOOKAHEAD` with a constrained outer pool scores 3.715 average, *worse* than plain entropy—showing that a 2-ply search with a restricted pool is not automatically better than a well-executed 1-ply search.

The crane paradox (Section 5) illustrates the core tension in concrete form: `crane` sits outside the top 15 by Shannon entropy yet achieves the highest ≤ 3-solve rate among all tested openers, because answer eligibility increases singleton bucket frequency and the ≤ 3-solve objective rewards many small buckets rather than equal expected partition size.

`FREQ_PRIOR` introduces a continuous word-frequency prior (Zipf scores, $\lambda = 0.06$) as a candidate bonus, replacing the binary common/uncommon split used by `BAYESIAN` and `PREFER_COMMON`. On the 193-word benchmark it achieves 3.389 average guesses and 59.1% $\leq$ 3-solve rate with zero failures—matching `HYBRID` on average and `MAXIMIZE_WIN_NOW` on the ≤ 3 rate. The marginal benefit of fine-grained frequency weighting appears small once a candidate-preference bonus of the right scale is already present.

Acknowledgments

The word list and answer pool were derived from the publicly available NYT Wordle source. Analysis was conducted using the `analysis.js` benchmark suite developed alongside the solver.

References

Berlekamp, E. R. (1964). Block coding with noiseless feedback. *PhD thesis, MIT*.

Bertsimas, D. and Paskov, A. (2025). An exact solution to Wordle. *Operations Research*, 73(3):1384–1394.

Huffman, D. A. (1952). A method for the construction of minimum-redundancy codes. *Proceedings of the IRE*, 40(9):1098–1101.

Sanderson, G. (2022). Solving Wordle using information theory. *3Blue1Brown video essay*. Available at `https://www.youtube.com/watch?v=v68zYyaEmEA`.

Speer, R. (2022). wordfreq: a Python library for looking up word frequencies from multiple language corpora. *Zenodo*. https://doi.org/10.5281/zenodo.7199437.

Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423.